

EVENT DRIVEN ARCHITECTURE



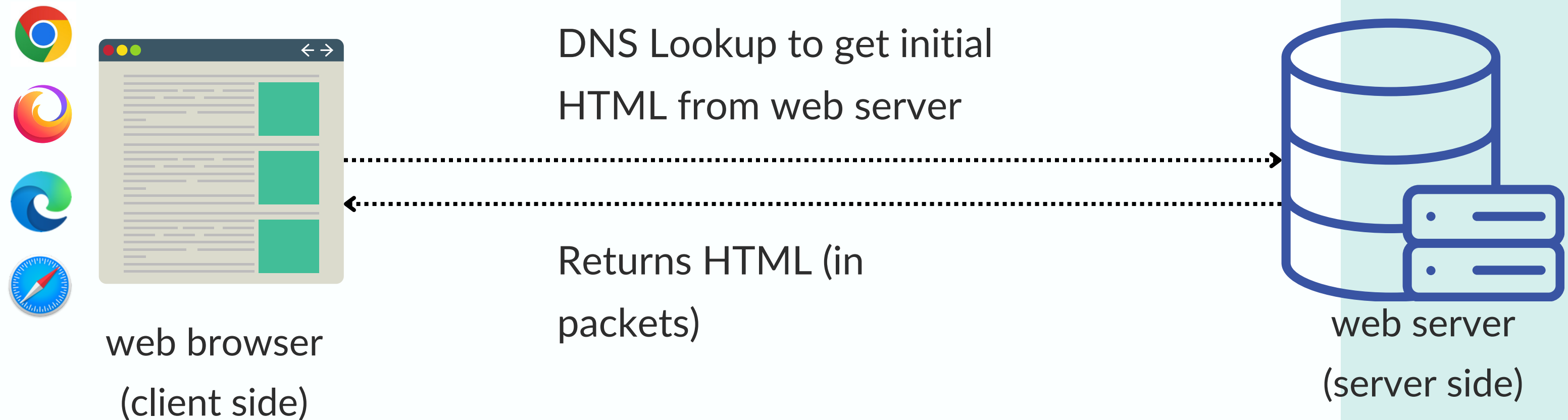
AGENDA

01 Recap

02 What is Event Driven Architecture?

03 WebSocket API w/ Express

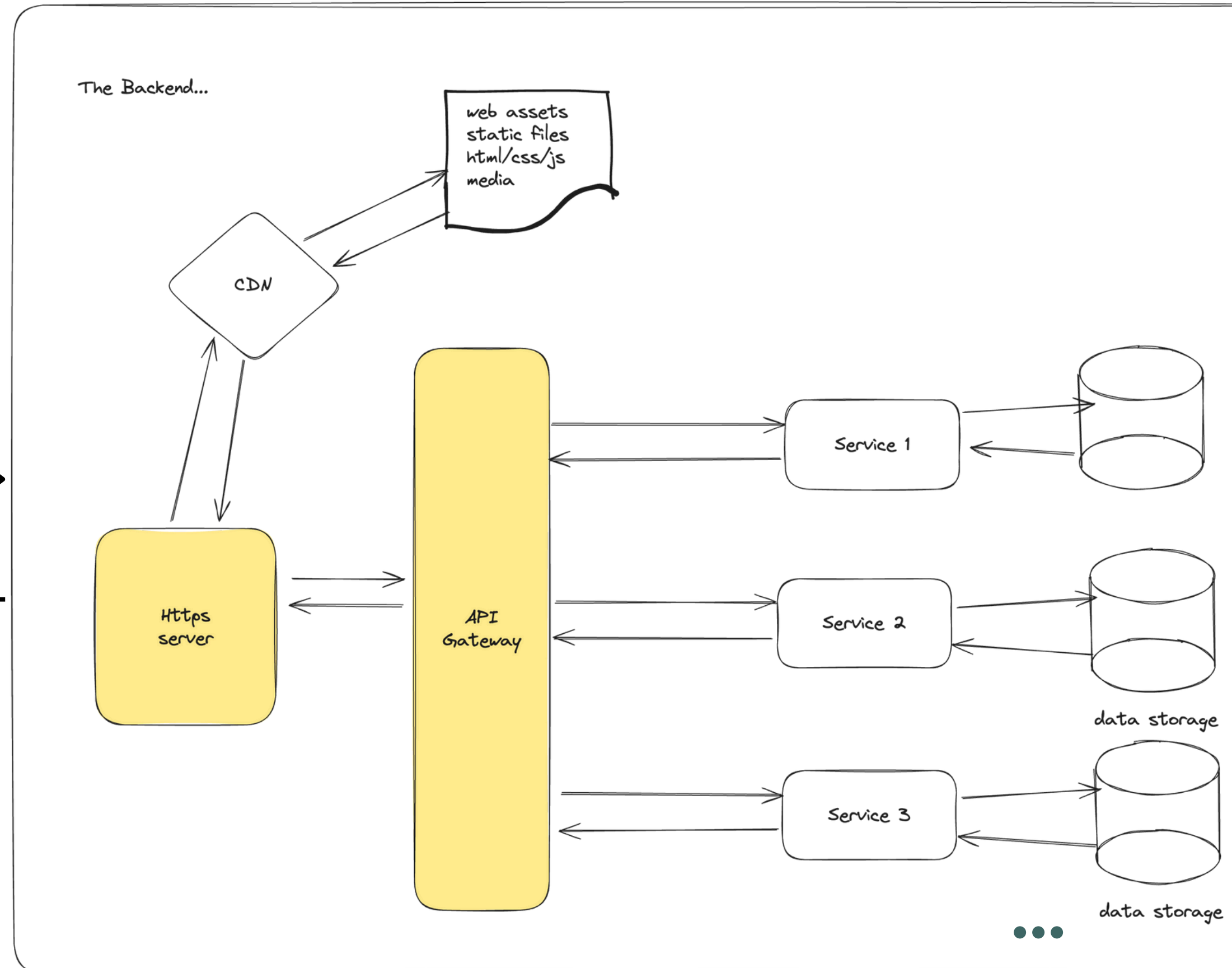
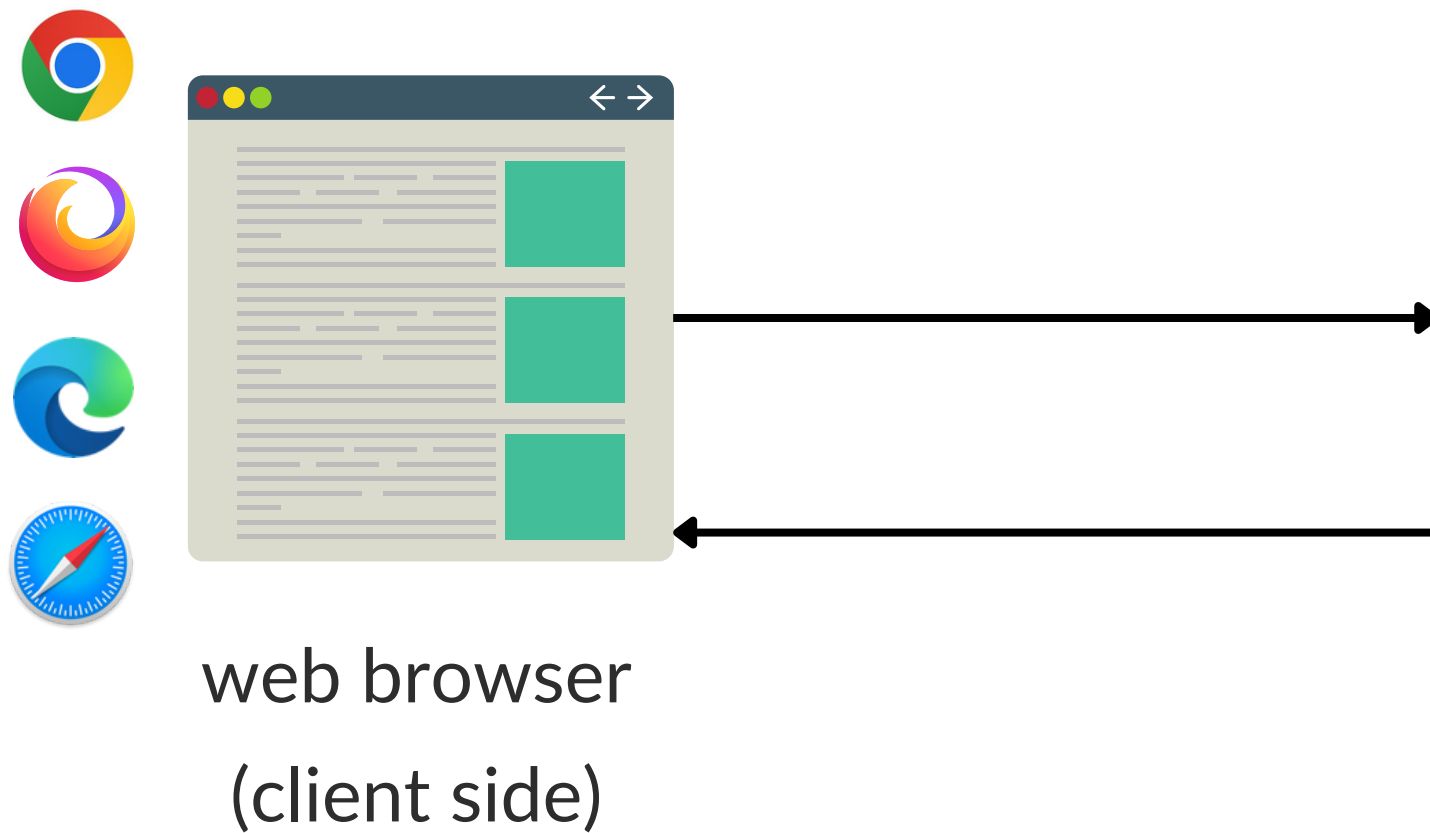
The web app that we all know...



Patterns for the BE

Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose

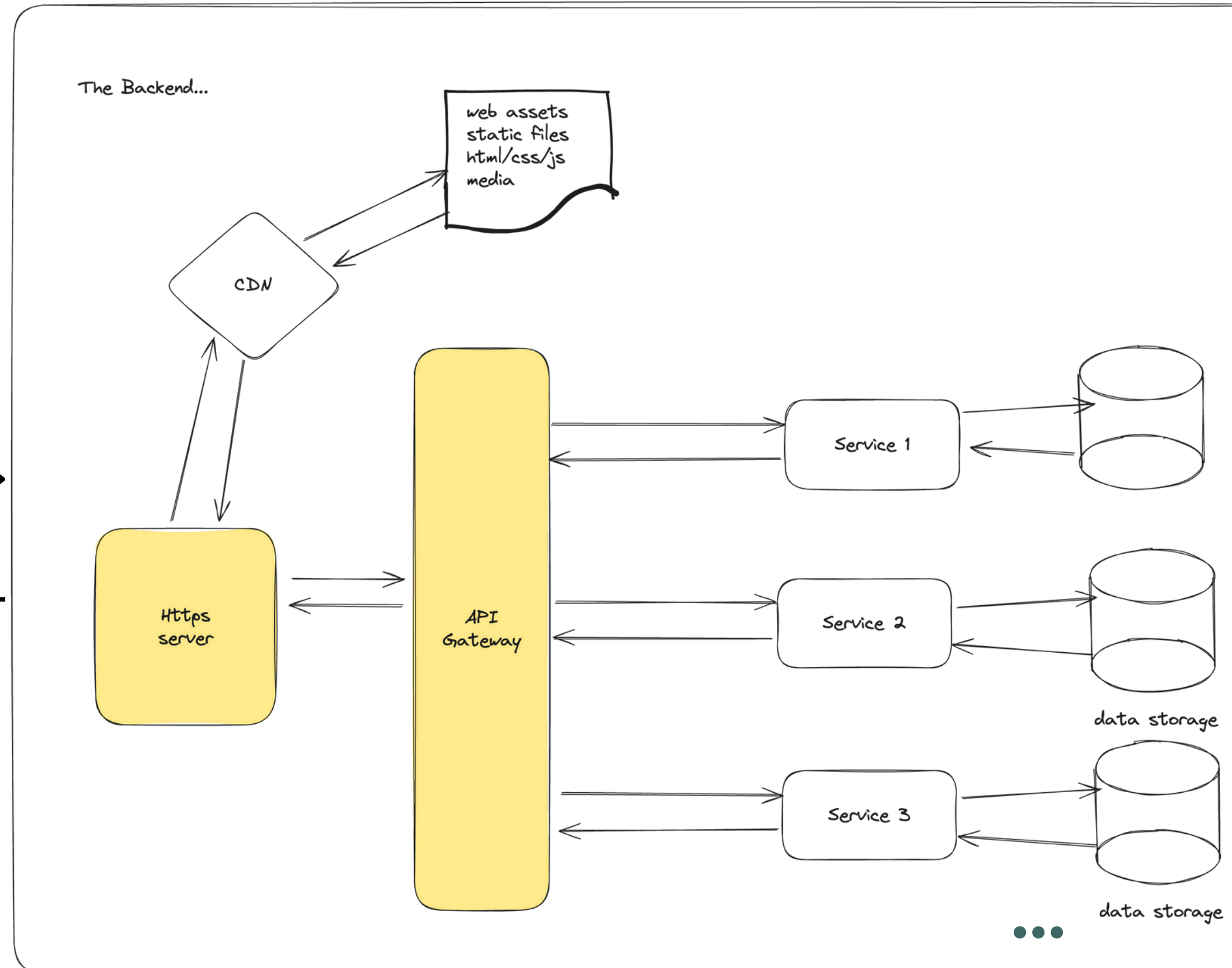
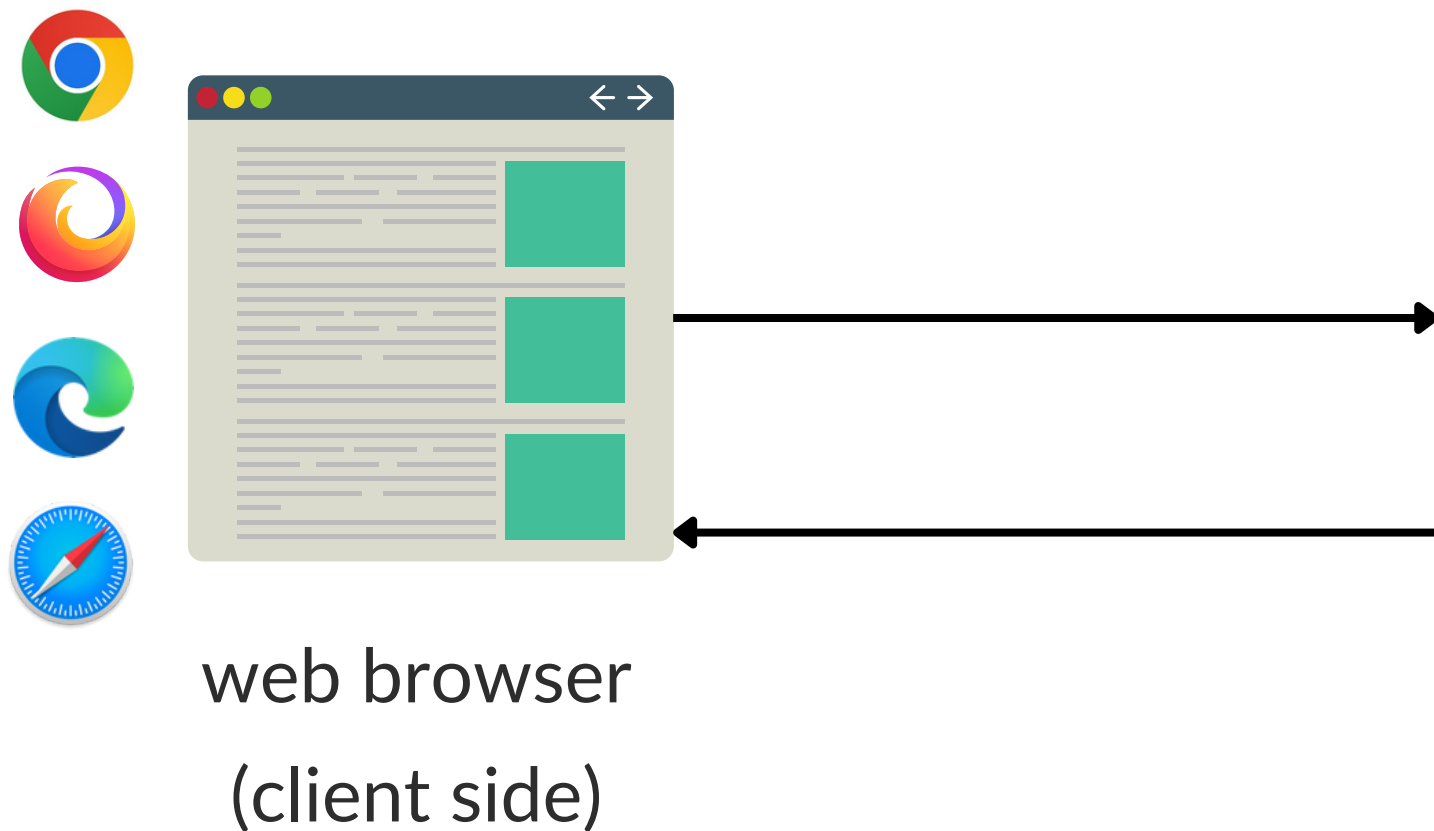


**WHEN DOES THE REQUEST-
RESPONSE NOT WORK?**

Patterns for the BE

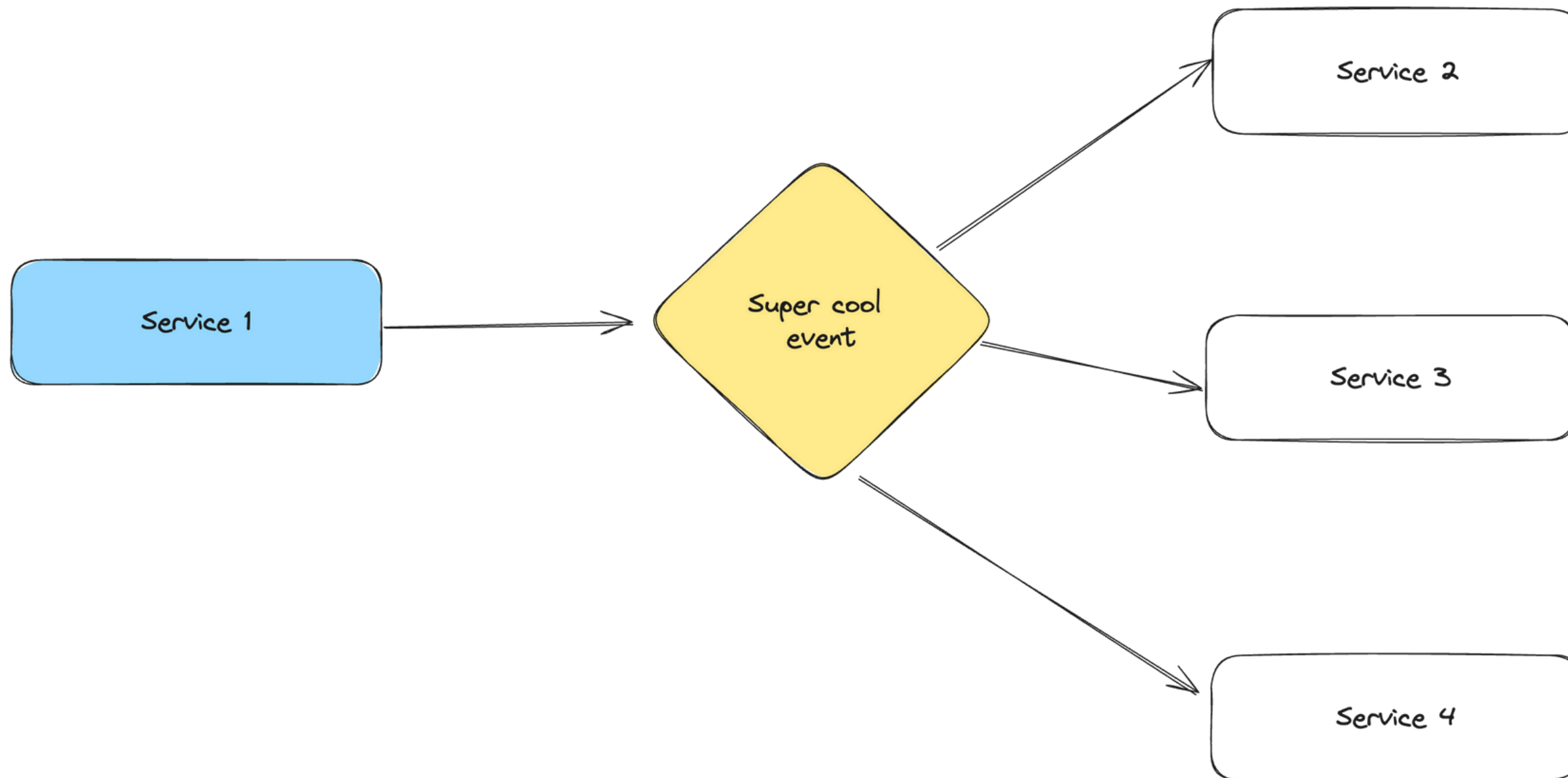
Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose



WHAT IS EVENT DRIVEN ARCHITECTURE (EDA)?

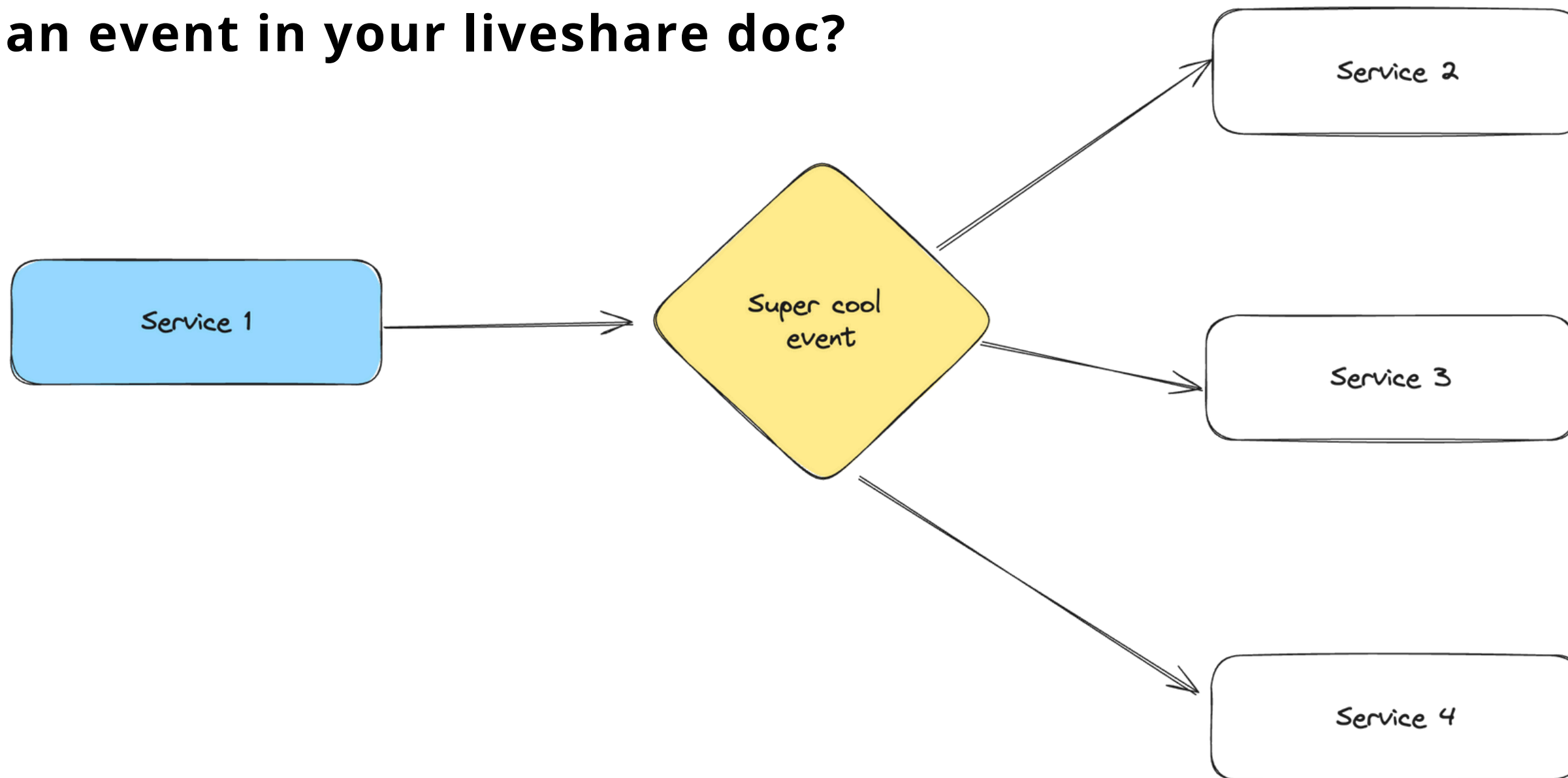
System design approach where we can have events created and sent to other services for processing



WHAT IS EVENT DRIVEN ARCHITECTURE (EDA)?

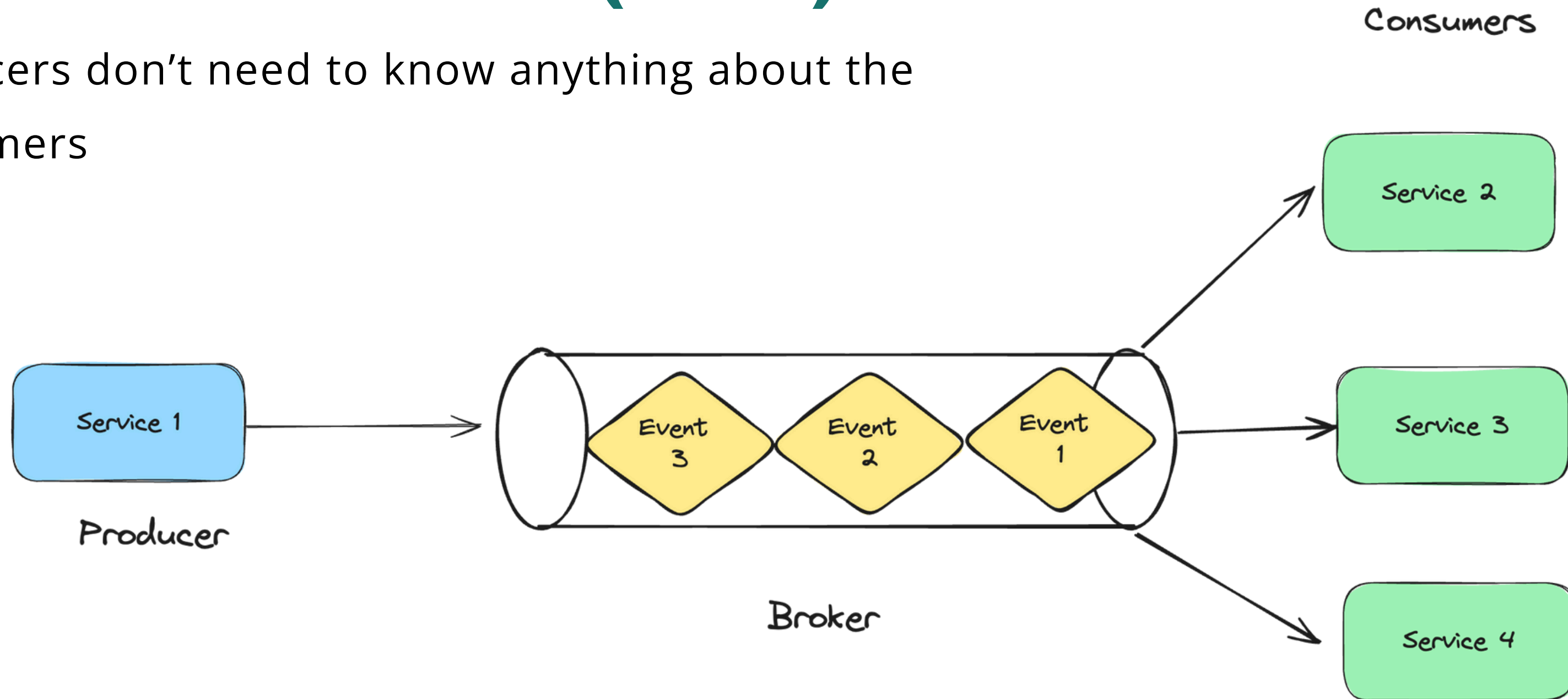
System design approach where we can have events created and sent to other services for processing

- **What is an event in your liveshare doc?**



WHAT IS EVENT DRIVEN ARCHITECTURE (EDA)?

Producers don't need to know anything about the consumers



MESSAGES VS EVENTS

Messages are sent through a queue to a specific location (ex: consumer) and the consumer needs to acknowledge that it received the message

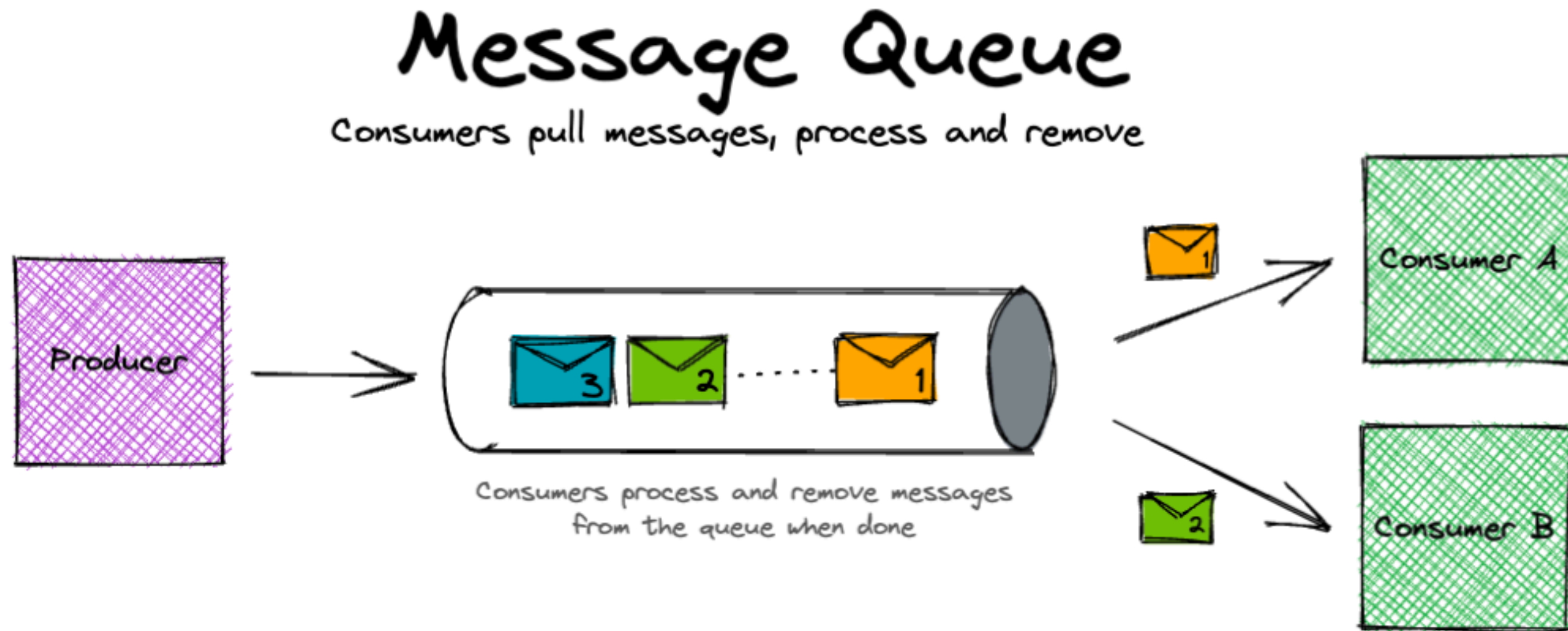


Image Source: <https://serverlessland.com/event-driven-architecture/visuals/message-queue-vs-event-broker>

EVENTS

Events are broadcast - consumers listen for the broadcasted event to process

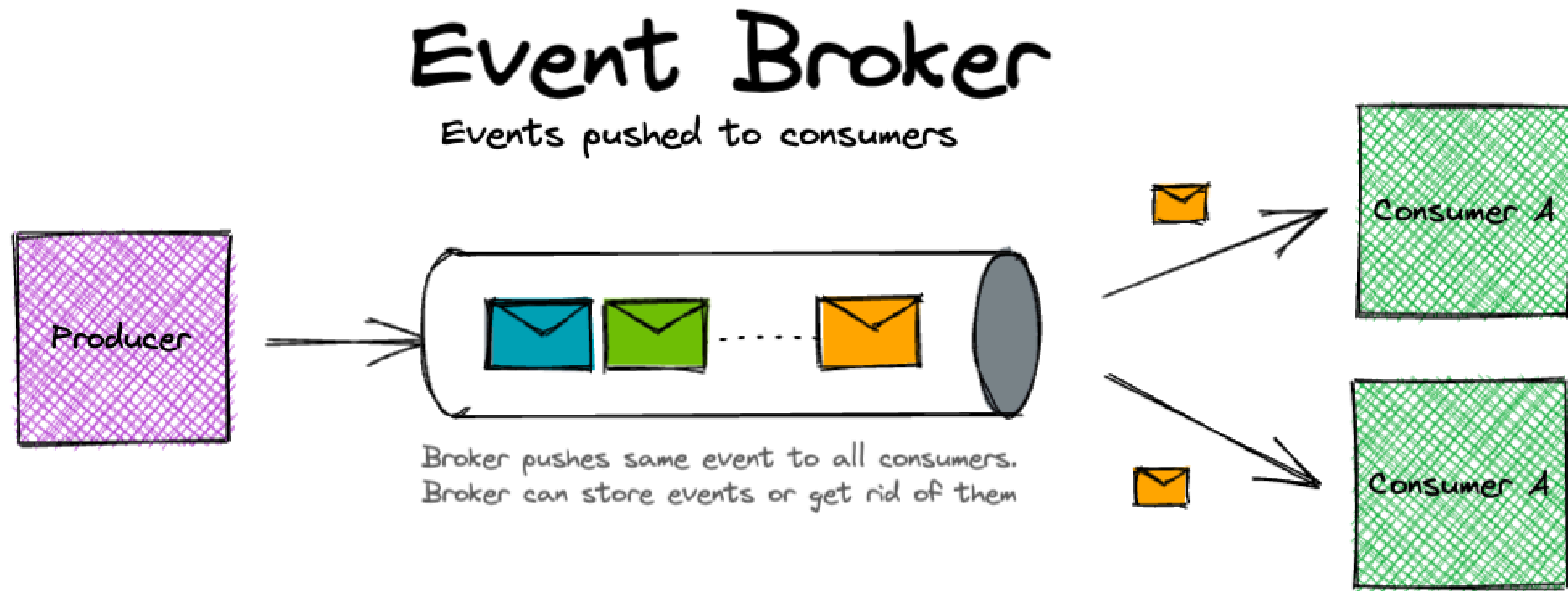
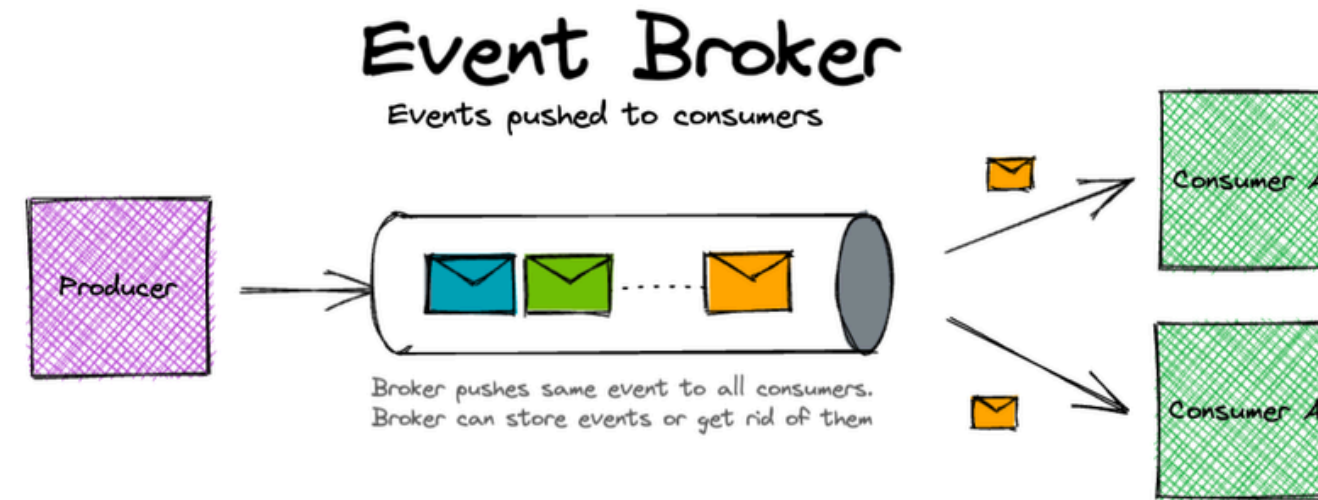


Image Source: <https://serverlessland.com/event-driven-architecture/visuals/message-queue-vs-event-broker>

WHICH IS MORE TIGHTLY COUPLED?

- a. Events
- b. REST APIs



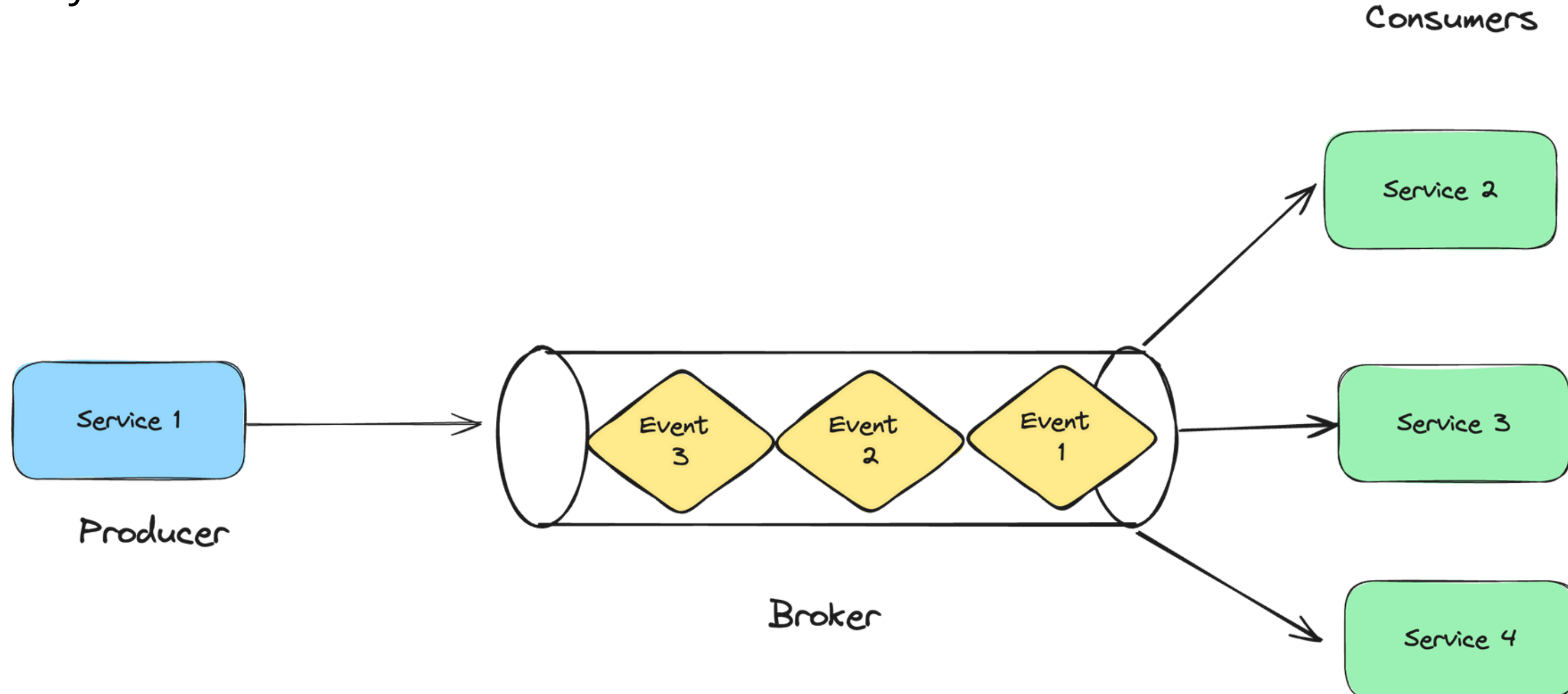
django

REST

framework

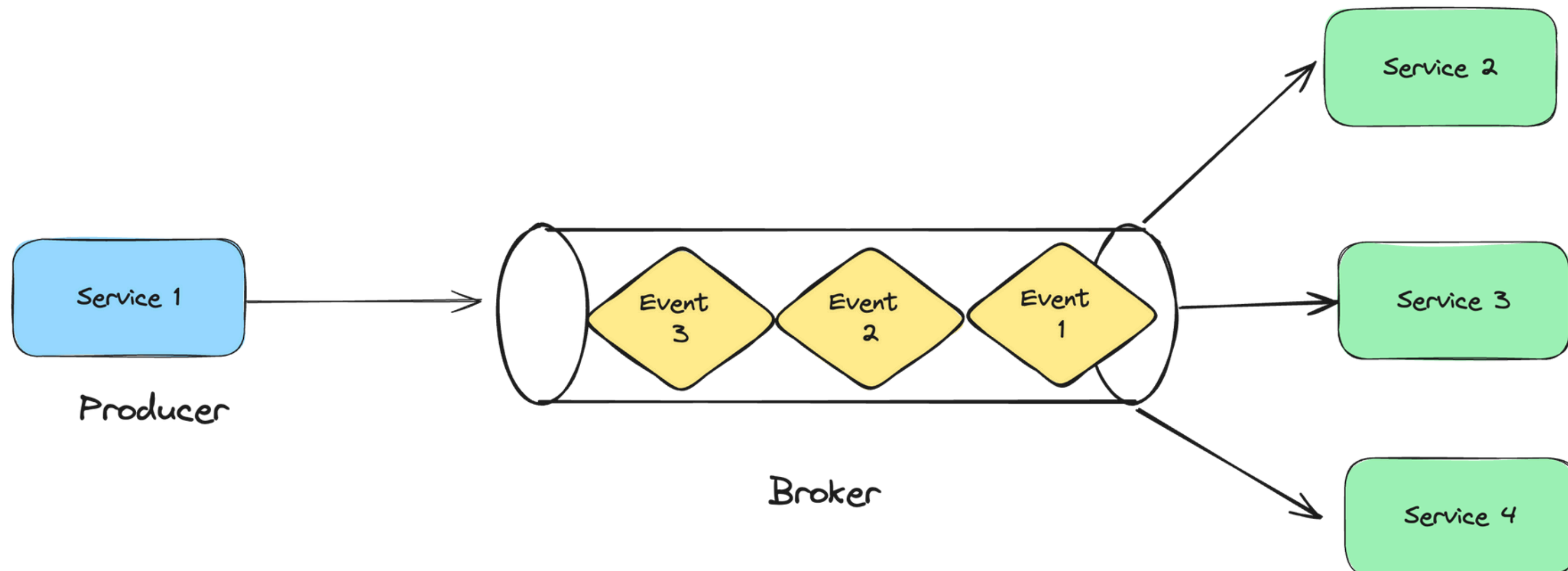
BENEFITS

- Decoupled
- Events are immutable so can be consumed by multiple consumers
- Events are also persistent
 - Why is this useful?



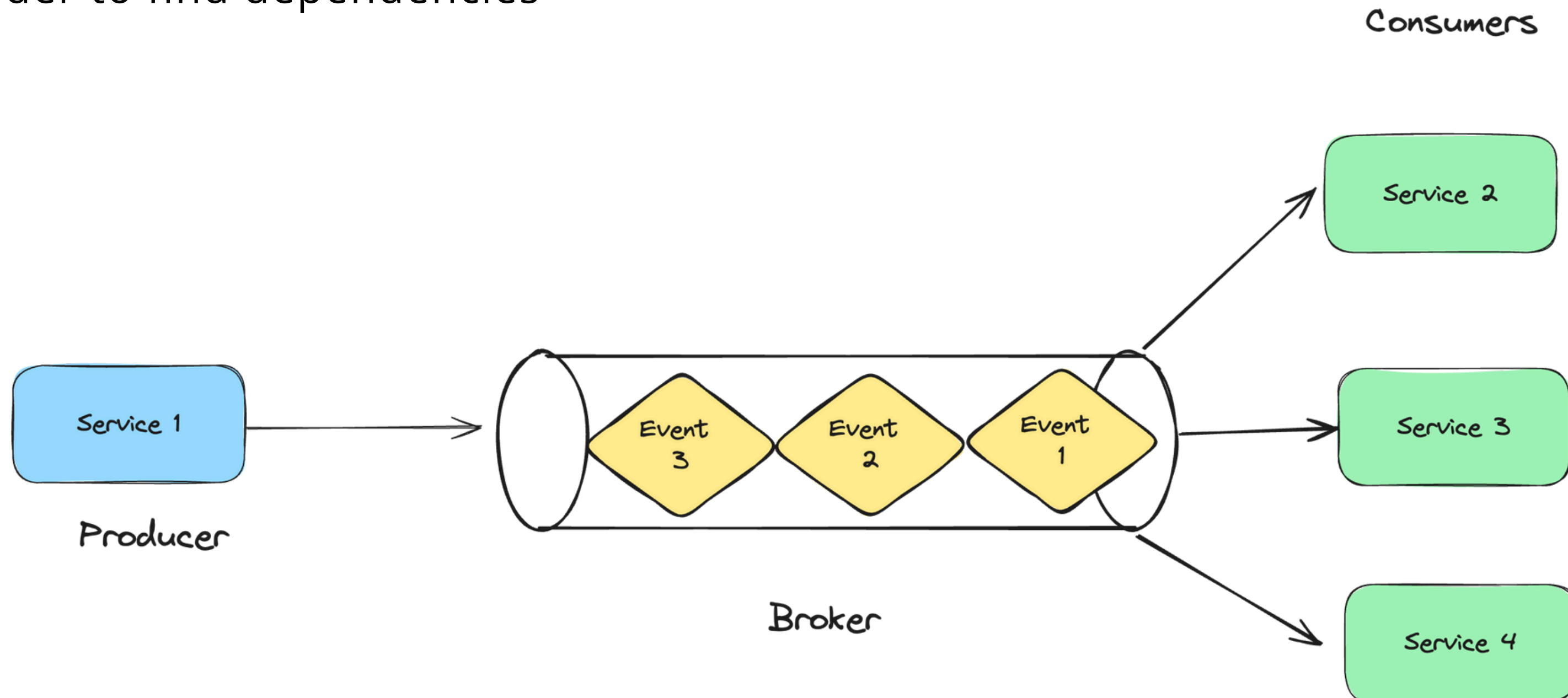
BENEFITS

- Decoupled
- Events are immutable so can be consumed by multiple consumers
- Events are also persistent
 - Robustness – service may be down but events will still be around when the service comes back online



DRAWBACKS

- Consistency – services might be out of sync
- Harder to find dependencies

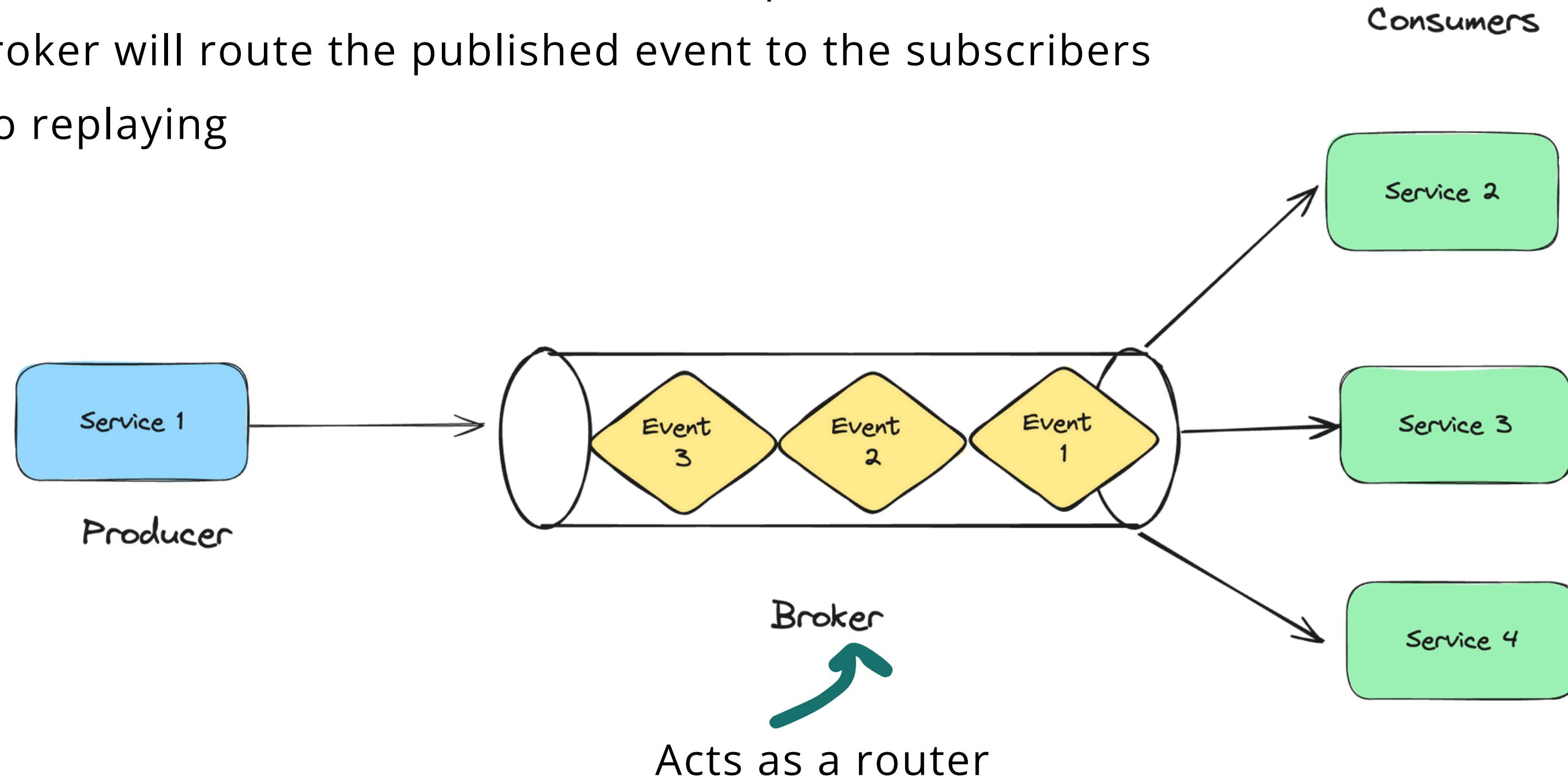


EDA PATTERNS

- Pub/sub
- Event streaming

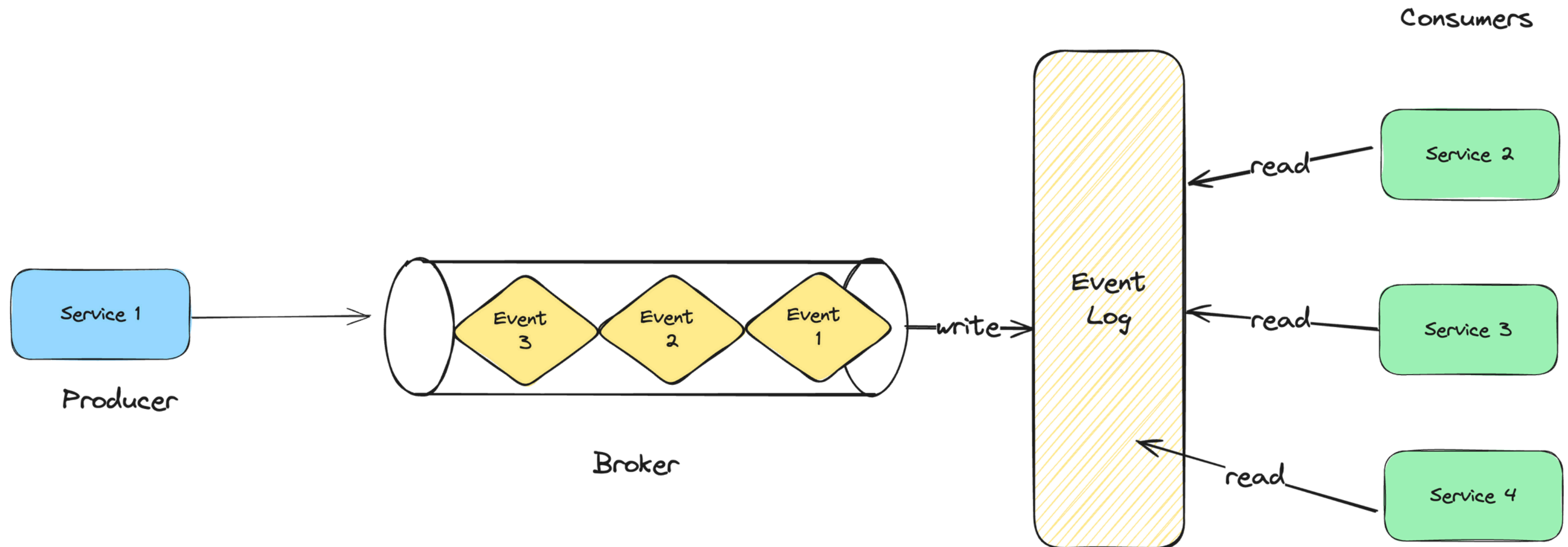
PUB/SUB

- Event broker tracks consumer subscriptions
- Broker will route the published event to the subscribers
- No replaying



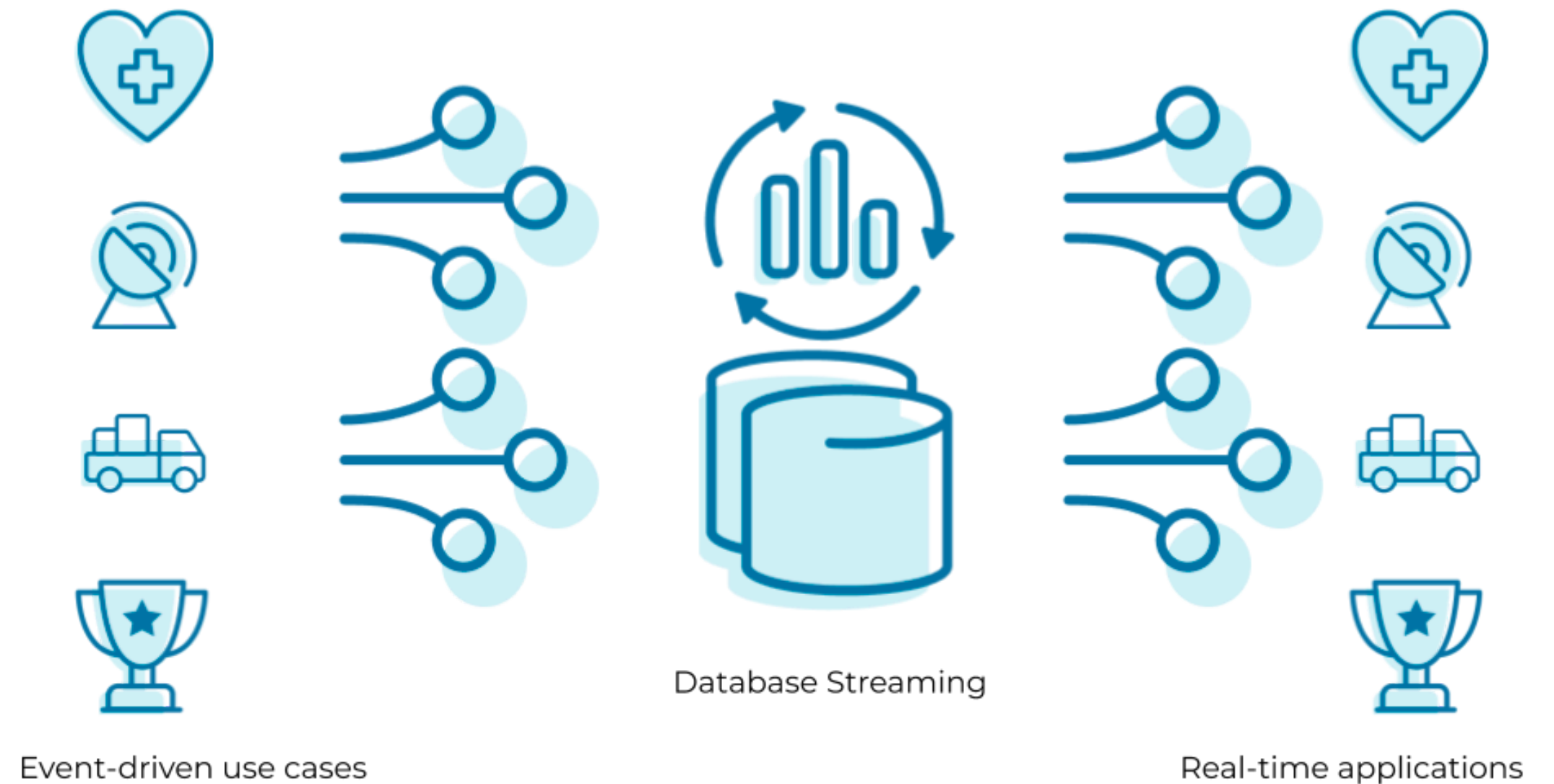
EVENT STREAMING

- Events are streamed (written) to a log that consumers can read at any point
 - Replay is possible



EVENT PROCESSING

- Consumer services can use events to perform actions
- Event streams usually ends up getting ingested by some data pipeline



REALTIME COMMUNICATION MODELS

- Polling
- Long polling
- Push

POLLING

- Client sends a request to server and server sends immediate response (regardless of if there's new info to send)
 - This is essentially our request-response model

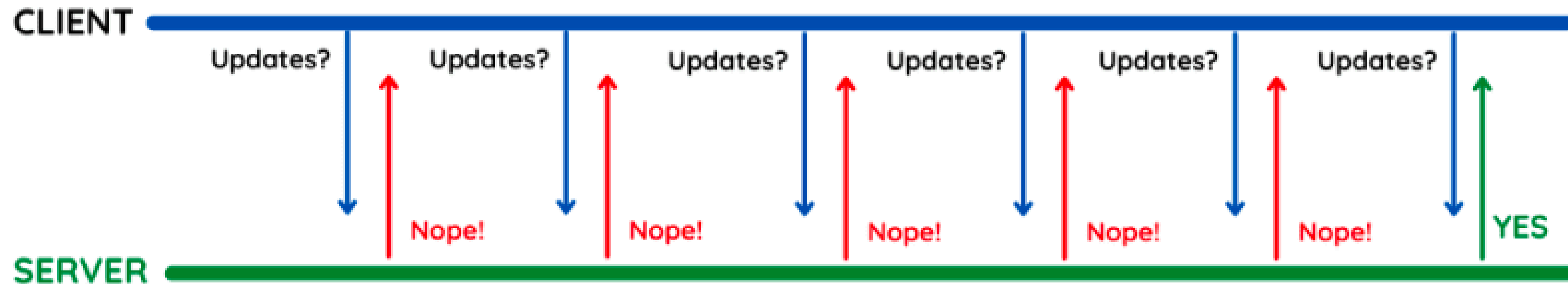


Image source: <https://dev.to/anubhavitis/push-vs-pull-api-architecture-1djo>

LONG POLLING

- Server chooses to hold the client connection open for as long as possible and then sends data once it's ready or time limit is reached

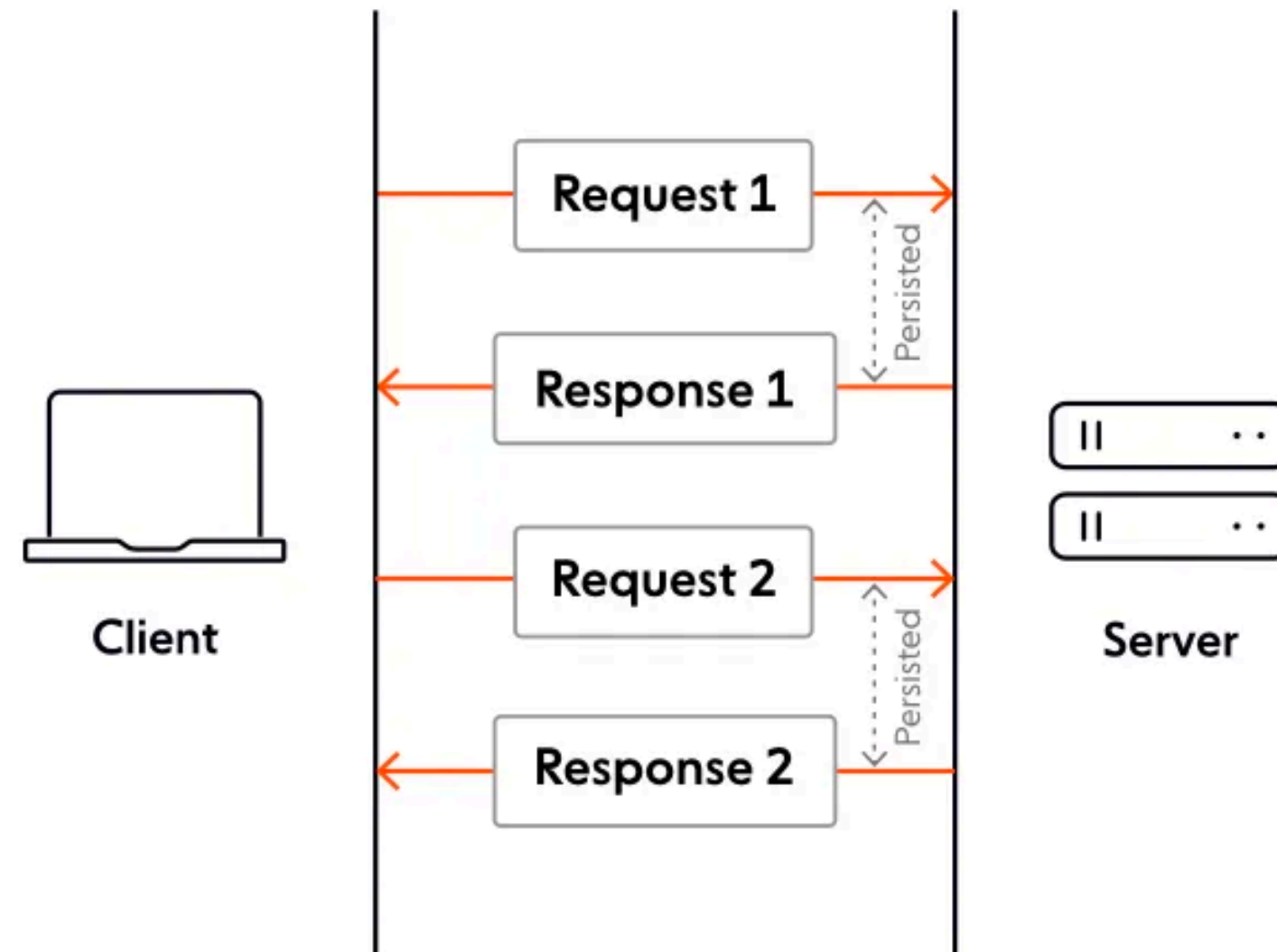


Figure 1.2: High-level overview of long polling

PUSH

- Client has a connection with the server that stays open
- Client waits for the messages that the server pushes
- Server keeps track of all the connected clients in a registry



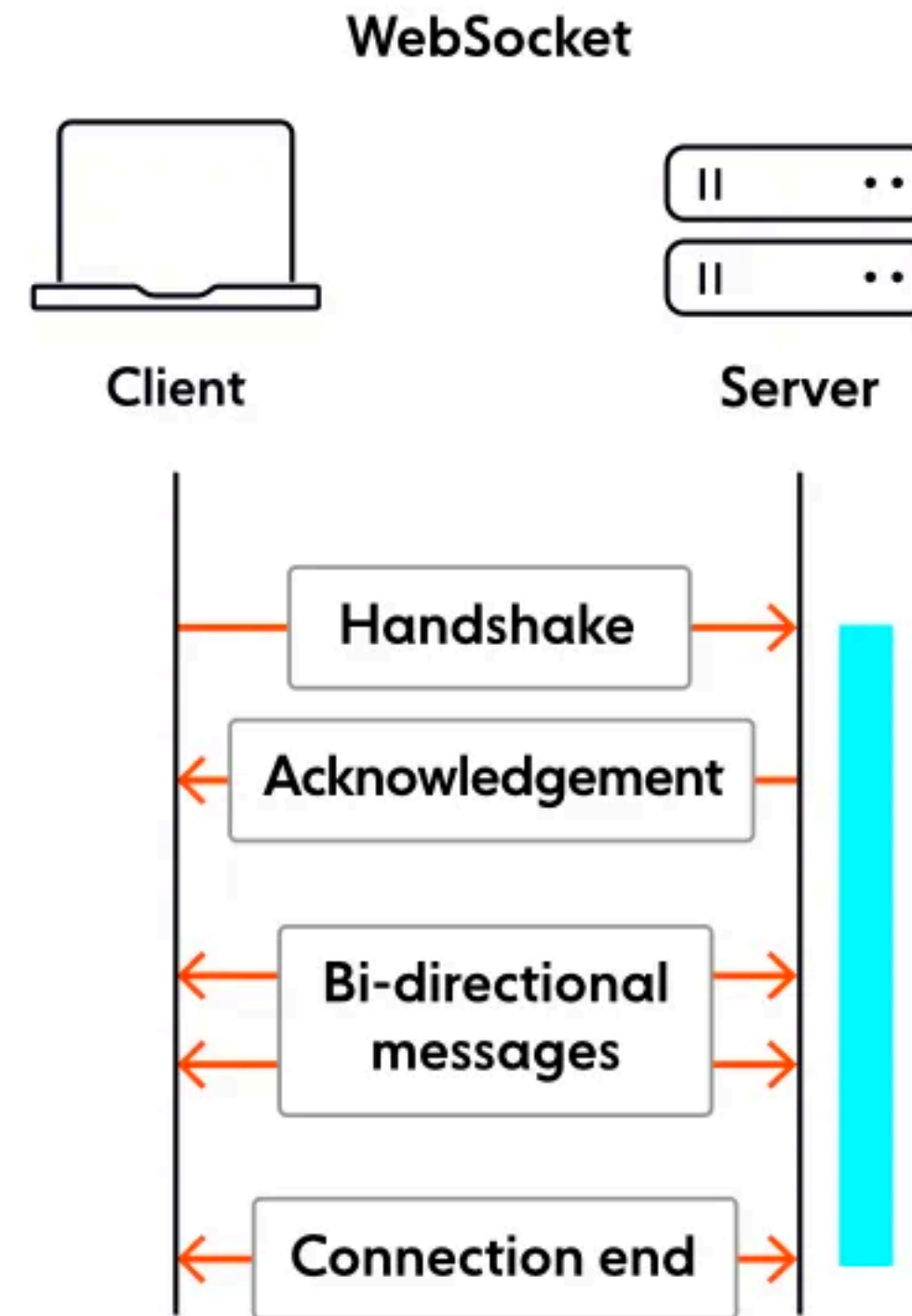
Image source: <https://dev.to/anubhavitis/push-vs-pull-api-architecture-1djo>

APPROACHES

- WebHooks
- Sockets
- Server Sent Events
- GraphQL Subscriptions
- gRPC

WEB SOCKETS

- Allows for bidirectional communication between the server and client
- Uses TCP (like HTTP is a layer that sits on top of TCP)
- Url uses the ws scheme
 - Ex: ws:// or wss://



WEB SOCKETS

STEP 1: THE HANDSHAKE

- GET request is sent from client to server
 - Specifies:
 - Connection: Upgrade
 - Upgrade: websocket

```
GET ws://websocket.example.com/ HTTP/1.1
Origin: http://example.com
Connection: Upgrade
Host: websocket.example.com
Upgrade: websocket
```

WEB SOCKETS

STEP 2: ACK

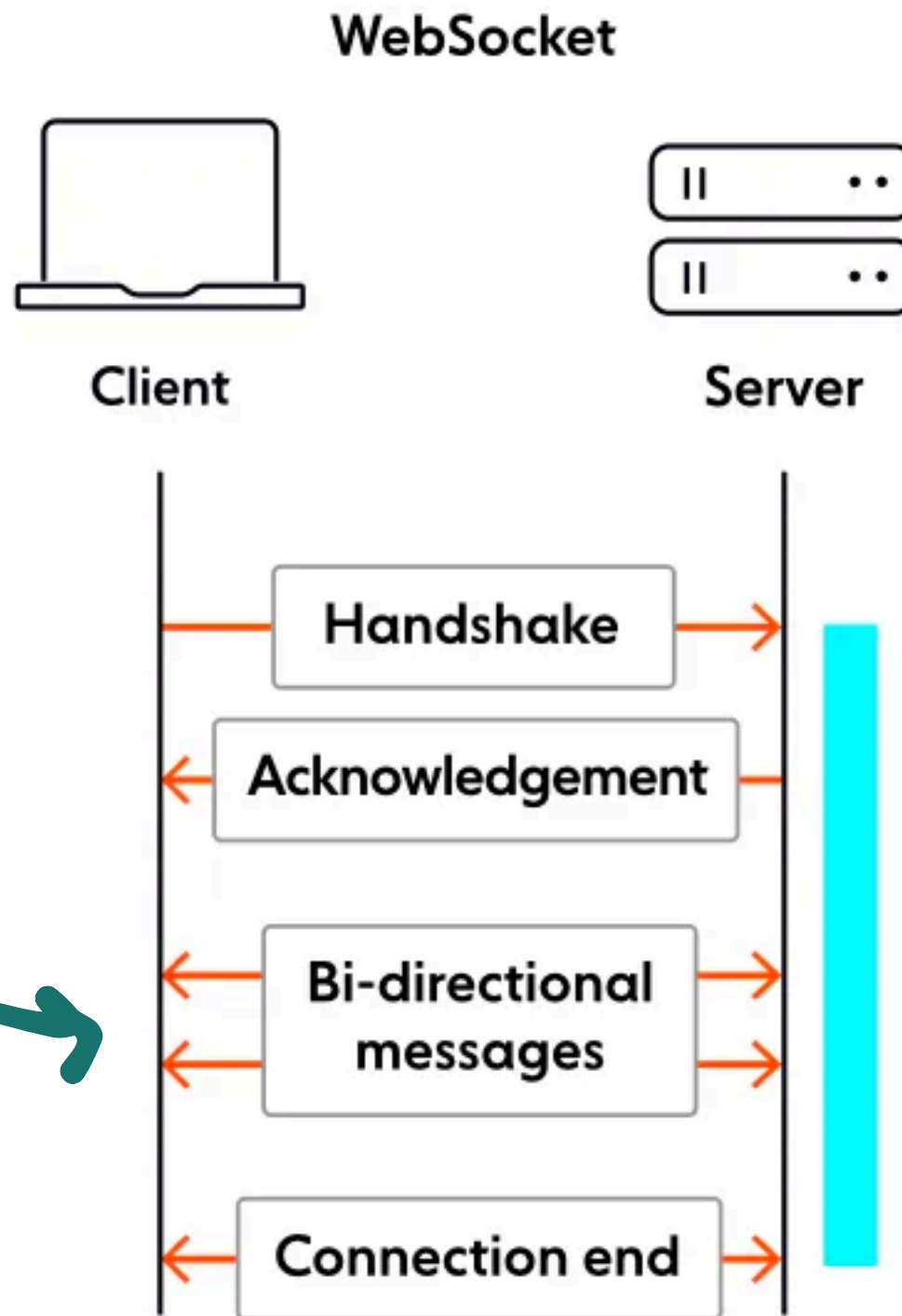
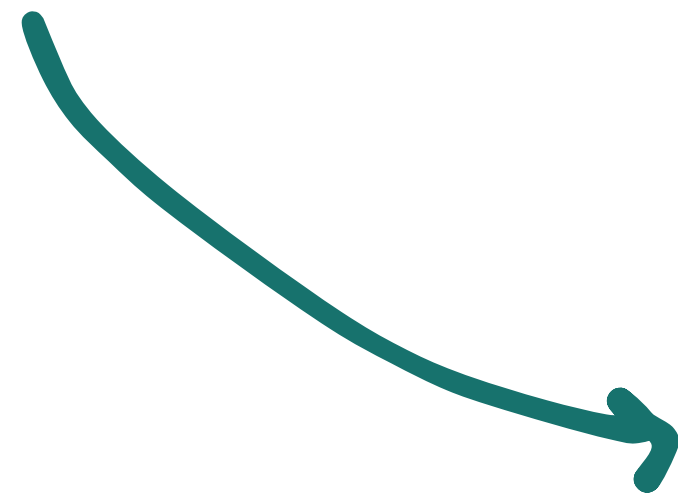
- Underlying HTTP connection gets replaced with WebSocket
 - WebSocket uses the same underlying TCP/IP connection

```
HTTP/1.1 101 WebSocket Protocol Handshake
Date: Wed, 16 Oct 2013 10:07:34 GMT
Connection: Upgrade
Upgrade: WebSocket
```

WEB SOCKETS

STEPS 3 & 4

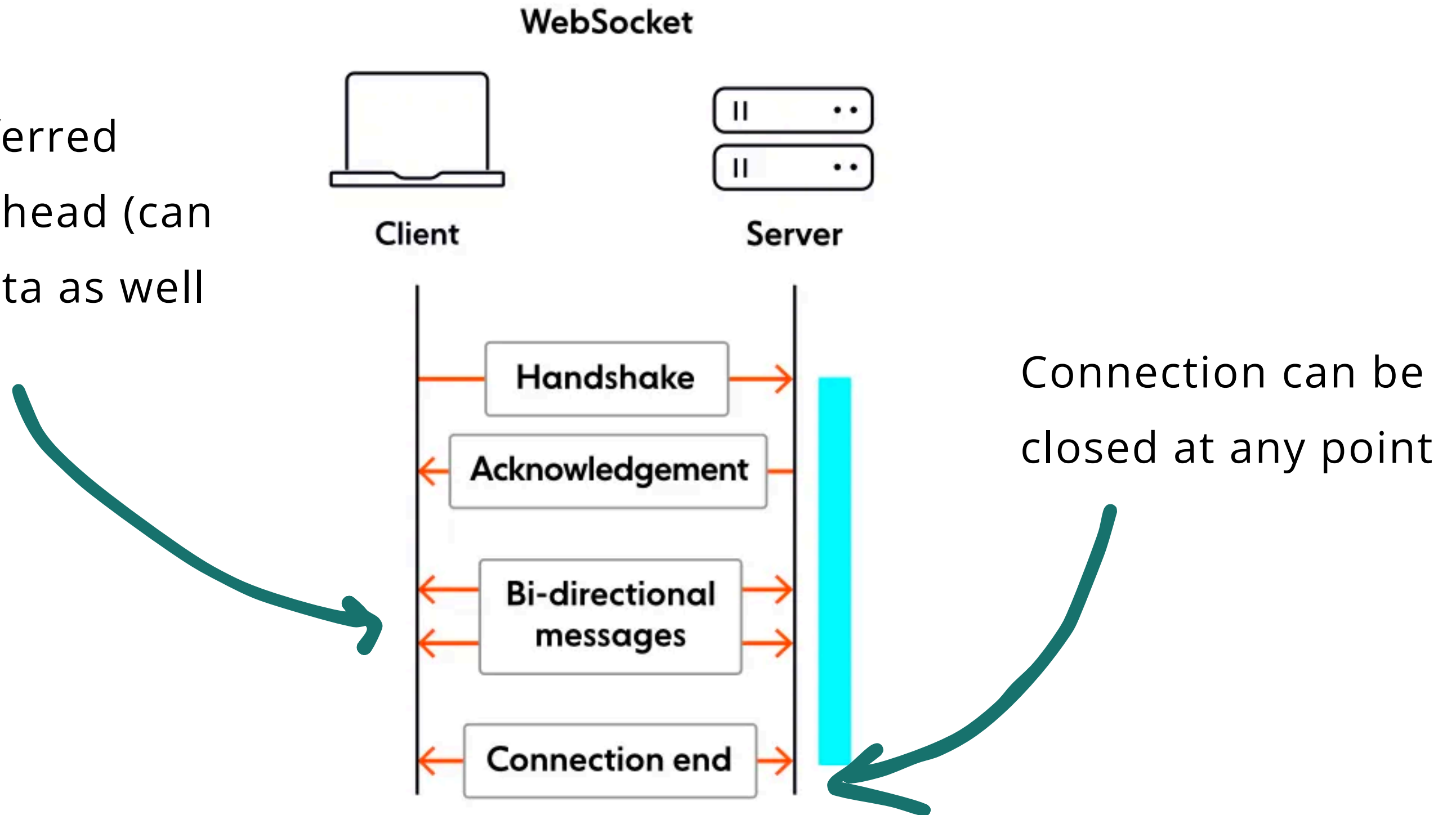
Data can be transferred without HTTP overhead (can transmit binary data as well as text)



WEB SOCKETS

STEPS 3 & 4

Data can be transferred without HTTP overhead (can transmit binary data as well as text)



**sketch out an
architecture diagram
for the liveshare doc
(using sockets)**

Creating our own (ultra simplistic) websocket server

<https://github.com/CS-396-Full-Stack-Software-Eng/ultra-basic-websocket-server>

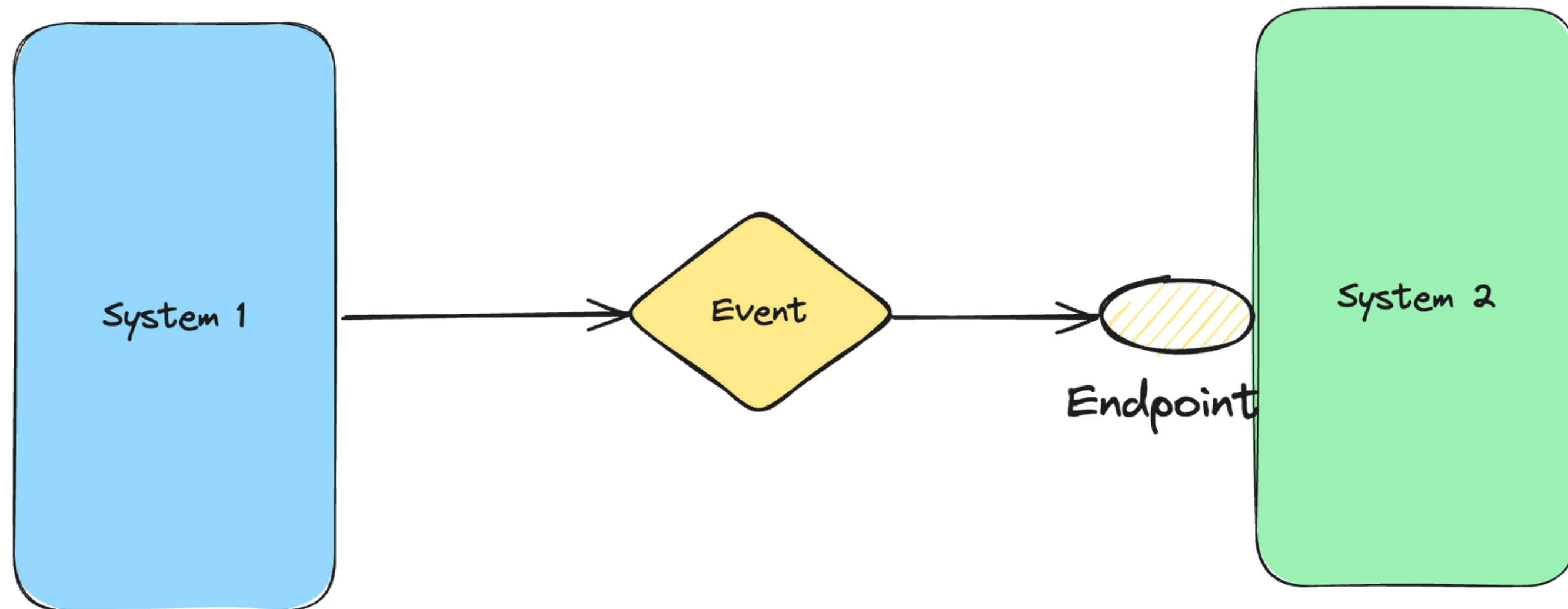
tinyurl.com/cs396socket

**what else would you
add to the architecture
diagram?**

WEBHOOKS

Server exposes an endpoint (hook) that the API server will then use to send event notifications

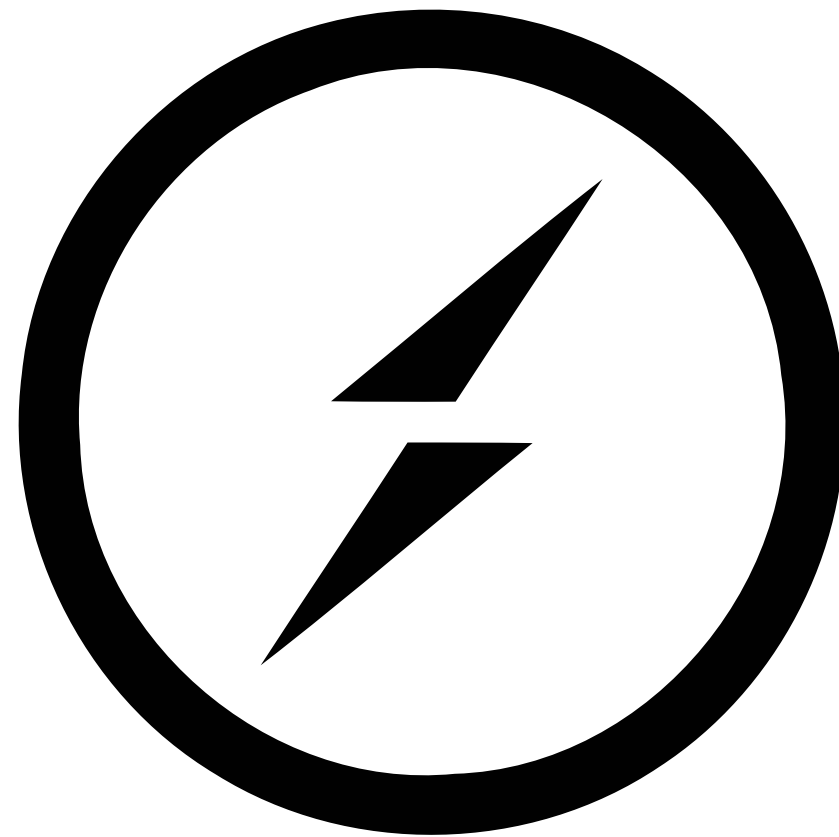
- Ex: GitHub webhooks
- Subscription is managed on the server side



WEBSOCKETS VS SOCKET.IO

Socket.IO is a library that has support for long polling, WebSocket, and WebTransport

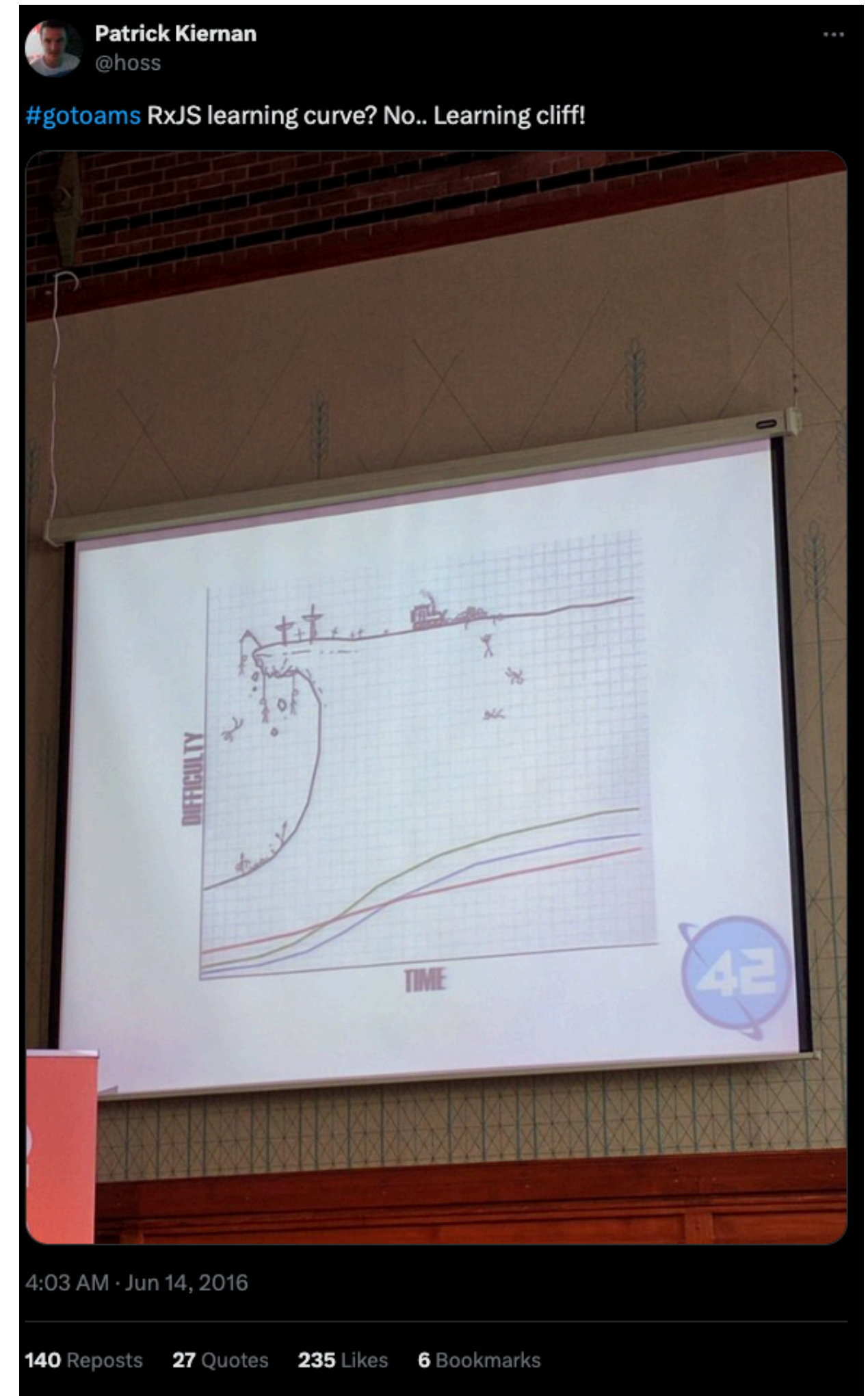
Downside: Socket.io is not as performant because of multiplexing and is generally not used when trying to build at scale



OTHER TOOLS: RXJS

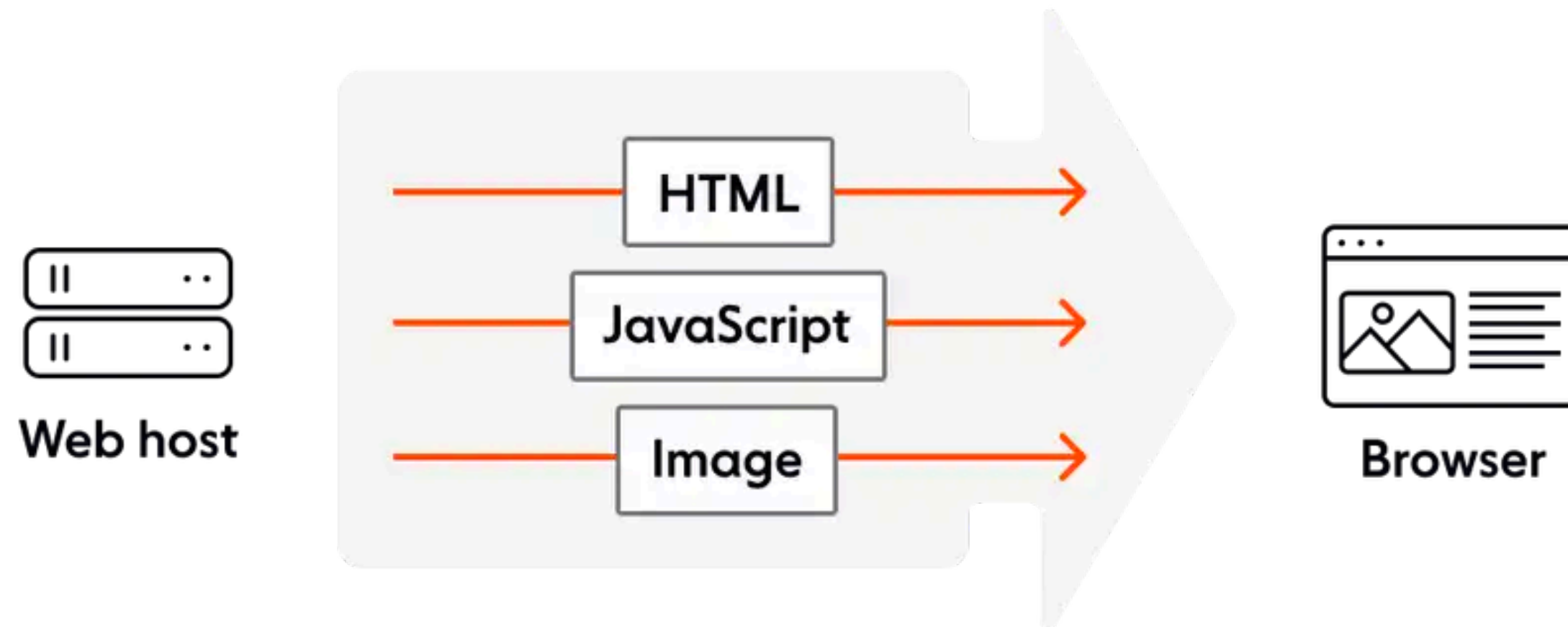
Library for reactive programming:

<https://www.learnrxjs.io/>



WEBSOCKETS VS WEBTRANSPORT

- WebTransport is very new and offers multiplexed realtime transport of data
 - Can create multiple streams over one single connection



WHY NOT USE WEBSOCKETS?

- Not very scalable
- Most apps just need to read info and don't need the bidirectionality

GRAPHQL SUBSCRIPTIONS

- Subscriptions allow your client to be notified whenever there are changes to the backend
- Standard doesn't specify which protocol to use but you can use WebSocket or HTTP

```
type Subscription {  
  commentAdded(postID: ID!): Comment  
}
```

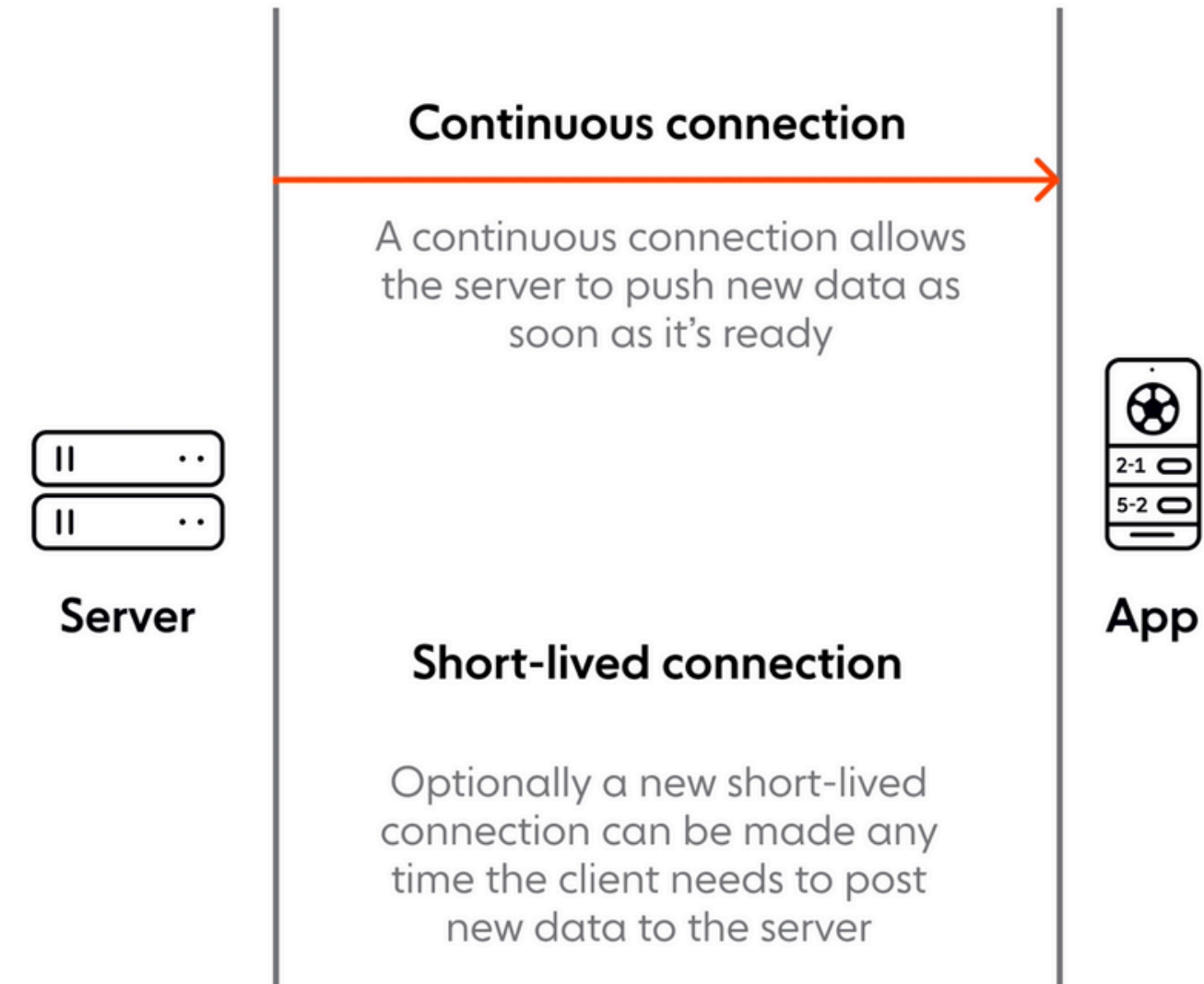
Schema

```
subscription OnCommentAdded($postID: ID!) {  
  commentAdded(postID: $postID) {  
    id  
    content  
  }  
}
```

Client side request

SERVER SENT EVENTS (SSE)

- Server pushes data to the browser when there's new info
- The client is in charge of subscribing
- Used by Shopify
 - Helpful for realtime notifications
- Uses HTTP
- Unidirectional (more secure)
- Has auto reconnection



gRPC

- Allows for bidirectional streaming

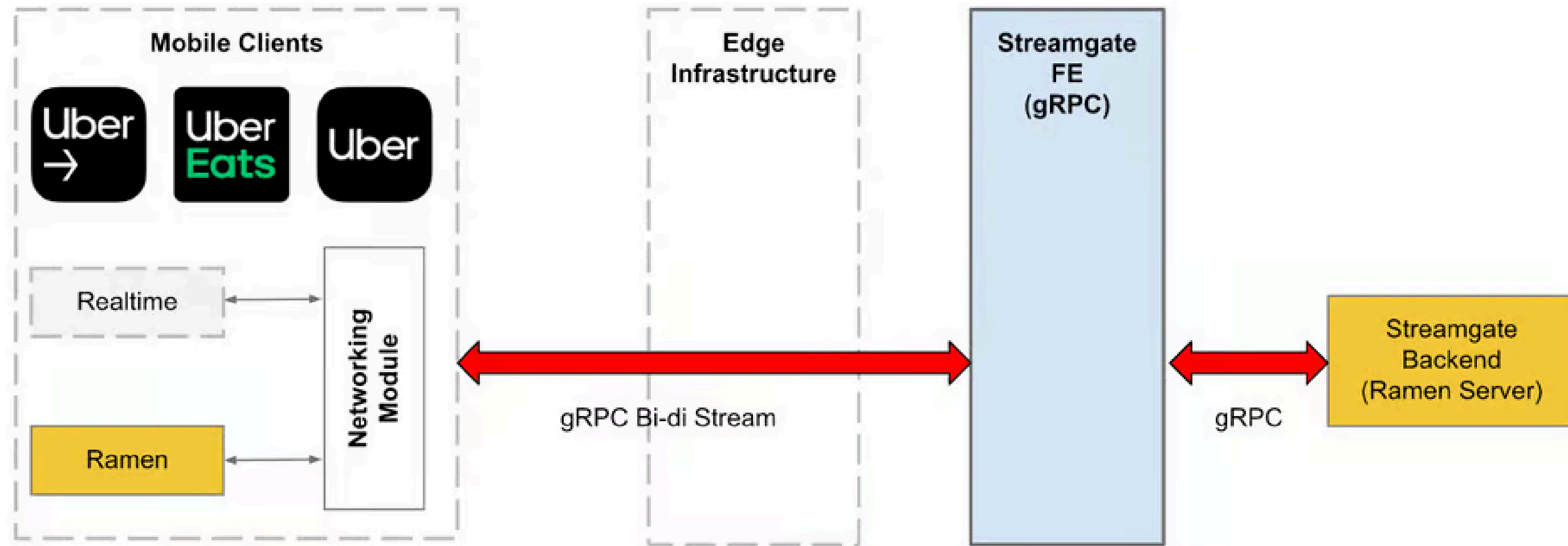


Image source: <https://www.uber.com/blog/ubers-next-gen-push-platform-on-grpc/>